

File System API and Monitoring

In this lab, you will practise the APIs related to files and directories. You will also practise writing and executing C programs to learn how the API works. Students who are not comfortable with writing code in C can review the provided C code in the Answers. They are encouraged to inspect and run this code and attempt to understand its functionality.

Contents

Exercise 1.....	2
Exercise 2.....	2
Exercise 3.....	2
Exercise 4.....	2
Exercise 5 (Advanced)	2
Exercise 6 (Advanced)	3

Exercise 1

Explain how the following system calls can be used for file handling: **open()**, **read()**, **write()**, and **close()**.

Write a C program that opens a file named "output.txt" for writing, prompts the user to enter a string of text, writes the entered text to the file, closes the file.

Demonstrate that your program works correctly.

Exercise 2

Write a C program that opens an existing file named "input.txt" for reading, reads the contents of the file, prints the contents to the console, closes the file.

Demonstrate that your program works correctly.

Use **strace** to identify the system calls used in your program.

What is the Open File Table (OFT)?

Exercise 3

Write a C program that opens an existing file named "source.txt" for reading and another file named "destination.txt" for writing, reads the contents of "source.txt", writes the contents to "destination.txt", closes both files.

Demonstrate that your program works correctly.

Use **strace** to identify the system calls used in your program.

Exercise 4

The **stat** command is used to display detailed information about a file or file system, such as its size, permissions, inode number, last access time, last modification time, and more. It provides a comprehensive overview of the metadata associated with a file or directory. You can use it by typing **stat** followed by the name of the file or directory you want to inspect.

Create one directory and one file, write to the file some text and then use the **stat** command to display detailed information.

Exercise 5 (Advanced)

Write a C program called **mystat.c**, which will be your own version of the command line program **stat**, which calls the **stat()** system call on a given file or directory. Print out file size, number of blocks allocated, and reference (link) count.

strace is a powerful diagnostic, debugging, and instructional utility for Linux. It is used to monitor the system calls and signals that a program receives, allowing you to inspect the behaviour of a program and troubleshoot issues related to system calls. When you run a program with **strace**, it intercepts and records the system calls that are made by the program and the signals that are received by the program. This can be useful for understanding how a program interacts with the operating system, identifying performance bottlenecks, diagnosing errors, and debugging issues such as improper file access or permission problems.

Run your program **mystat** with **strace**.

Exercise 6 (Advanced)

Write a C program called **mys.c** that lists files in the given directory. When called without any arguments, the program should just print the file names. When invoked with the **-l** flag, the program should print out information about each file, such as the owner, group, and permissions, obtained from the **stat()** system call. The program should take one additional argument, which is the directory to read, e.g., **mys -l directory**. If no directory is given, the program should just use the current working directory. Useful interfaces: **stat()**, **opendir()**, **readdir()**